OPIPE-04: Cardinality Management

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 4 of 6 | **Created:** March 2026 | **Last Updated:** 10/15/2026

Controlling Dimension Explosion Across All Scopes

Cardinality — the number of unique combinations of dimension values in your data — is the single biggest cost and performance driver in observability. High cardinality slows queries, inflates storage, and can cause metric data to be dropped entirely. OpenPipeline is the first line of defense: it lets you normalize, reduce, and manage cardinality **before** data reaches Grail.

This notebook covers cardinality concepts and practical reduction strategies that apply across all OpenPipeline scopes.

For bucket-level cost controls, see **OPLOGS-04: Buckets & Data Governance**. For query optimization on stored data, see **SPANS-08: Cost-Efficient DQL Queries**.

Table of Contents

1. What Is Cardinality and Why It Matters
2. High-Cardinality Fields by Scope
3. Detecting Cardinality Problems
4. Reduction Strategy 1: Attribute Removal
5. Reduction Strategy 2: Value Normalization
6. Reduction Strategy 3: Grouping and Bucketing
7. Reduction Strategy 4: Hashing for Privacy
8. Metric Dimension Guardrails
9. Summary

10. Next Steps
11. References

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail
Permissions	`storage:logs:read`, `storage:spans:read`, `storage:metrics:read`
Recommended	**OPIPE-01** (multi-scope architecture) and **OPIPE-03** (metric extraction)

1. What Is Cardinality and Why It Matters

Cardinality is the number of unique combinations of values across the dimensions of your data.

A Simple Example

A metric with dimensions `dt.service.name` (50 services) and `http.route` (200 routes) produces:

```
50 x 200 = 10,000 unique time series
```

Add `http.response.status_code` (5 values) and `k8s.namespace.name` (10 namespaces):

```
50 x 200 x 5 x 10 = 500,000 unique time series
```

Now add `user.id` (100,000 unique users):

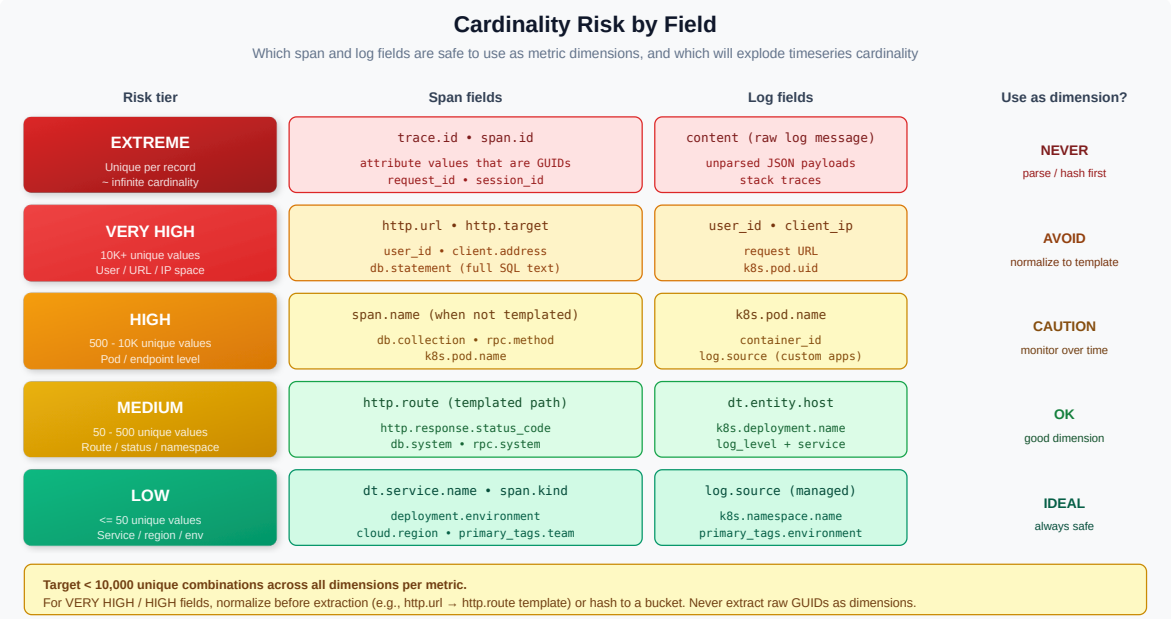
```
50 x 200 x 5 x 10 x 100,000 = 50,000,000,000
```

That is far past the 1 million per-metric dimension limit Metrics Classic enforces, and exactly the volatile-dimension configuration that Grail *"may restrict or reject"* (row below).

Impact of High Cardinality

Area	Impact
Metric storage	Each unique dimension combination is a separate time series. Millions of series consume significant storage.
Query performance	Queries against high-cardinality data scan more rows, hit scan limits faster, and return slowly.
Metric ingestion	Metrics Classic enforces a 1 million per-metric dimension limit; on Grail, *"Dynatrace therefore may restrict or reject metric configurations"* built on volatile dimensions such as timestamps or unique IDs (Metric limits (DT docs)).
Cost	More stored data = higher DPS consumption. High-cardinality dimensions multiply cost multiplicatively, not additively.

2. High-Cardinality Fields by Scope



Different scopes have different cardinality offenders. Know which fields to watch:

Spans

Field	Typical Cardinality	Risk
`trace.id`	Unique per trace	**Extreme** — never use as metric dimension
`span.id`	Unique per span	**Extreme**
`http.url`	Unique per request (includes query params)	**Very High**
`span.name`	Hundreds to thousands	**High** if operations include dynamic segments
`http.route`	Tens to hundreds	**Medium** — usually safe
`dt.service.name`	Tens	**Low** — always safe

Logs

Field	Typical Cardinality	Risk
`content`	Unique per log line	**Extreme** — never use as dimension
`log.source`	Tens	**Low**
`k8s.pod.name`	Hundreds (includes replica IDs)	**High** — pod names are ephemeral
`k8s.namespace.name`	Tens	**Low**
`dt.entity.host`	Tens to hundreds	**Medium**

Metrics (Extension-Ingested)

Field	Typical Cardinality	Risk
`dt.entity.process_group_instance`	Hundreds	**High** — process instances are ephemeral

Field	Typical Cardinality	Risk
Custom extension dimensions	Varies	**Depends** — review each extension's schema

3. Detecting Cardinality Problems

Run these queries to find high-cardinality dimensions in your environment.

```
// Detect high-cardinality log fields
fetch logs, from:-1h
| summarize total = count(),
    unique_sources = countDistinct(log.source),
    unique_buckets = countDistinct(dt.system.bucket),
    unique_hosts = countDistinct(dt.entity.host),
    unique_pods = countDistinct(k8s.pod.name),
    unique_namespaces = countDistinct(k8s.namespace.name)
```

```
// Detect high-cardinality span fields
fetch spans, from:-1h
| summarize total = count(),
    unique_services = countDistinct(dt.service.name),
    unique_operations = countDistinct(span.name),
    unique_routes = countDistinct(http.route),
    unique_traces = countDistinct(trace.id)
```

```
// Find span.name values with dynamic segments (cardinality risk)
// Look for names containing UUIDs, numeric IDs, or long paths
fetch spans, from:-1h
| summarize span_count = count(), by:{span.name}
| filter span_count &lt; 10
| sort span.name asc
| limit 30
```

4. Reduction Strategy 1: Attribute Removal

The simplest approach: remove fields you do not need before they reach storage or metric extraction.

When to Use

- Fields that are unique per record and never queried (session tokens, request IDs used only for debugging)
- Redundant fields (both `http.url` and `http.route` exist — keep `http.route`, drop `http.url`)
- Verbose attributes from third-party instrumentation that add no value

OpenPipeline Configuration

In the **Processing** stage of your pipeline, add a **Remove fields** processor:

Fields to Remove	Scope	Rationale
`http.url` (when `http.route` exists)	Spans	URL has query params; route is the pattern
Captured header attributes (`http.request.header.<name>`)	Spans	Request headers are rarely needed post-ingestion — list each attribute by name
`thread.name`, `thread.id`	Logs	Per-thread IDs are high-cardinality noise

5. Reduction Strategy 2: Value Normalization

Replace high-cardinality values with lower-cardinality patterns at ingestion.

URL Path Normalization

Before (high cardinality)	After (low cardinality)
`/users/12345/orders/67890`	`/users/{id}/orders/{id}`
`/api/v2/entities/HOST-ABC123`	`/api/v2/entities/{entity_id}`
`/search?q=dynatrace+monitoring`	`/search`

Status Code Grouping

Before	After
`200`, `201`, `204`	`2xx`
`301`, `302`, `304`	`3xx`
`400`, `401`, `403`, `404`, `422`	`4xx`
`500`, `502`, `503`	`5xx`

Pod Name Normalization

Kubernetes pod names include replica identifiers that are ephemeral:

Before	After
`checkout-service-7b4f6d9c8-x2k4q`	`checkout-service`
`payment-worker-5d8f7a3b2-m9n1p`	`payment-worker`

Don't rewrite `k8s.pod.name` itself — other views and entity links rely on it. Use the workload field Kubernetes monitoring already supplies (`k8s.deployment.name`, or `k8s.workload.name` where it is set) as the metric dimension. On the validation tenant, `k8s.deployment.name` was set on 767,490 of 913,405 pod log records in an hour (131 pods → 54 deployments). Where neither is present, derive one in a **DQL** processor in the Processing stage with `parse` and store it in a new field.

```
// Check: How many unique pod names vs. deployment names?
fetch logs, from:-1h
| filter isNotNull(k8s.pod.name)
| summarize unique_pods = countDistinct(k8s.pod.name),
    unique_deployments = countDistinct(k8s.deployment.name)
| fieldsAdd cardinality_reduction = if(unique_deployments &gt; 0,
    then: toString(round(toDouble(unique_pods) / toDouble(unique_deployments),
decimals: 1)) ,
    else: "N/A")
```

6. Reduction Strategy 3: Grouping and Bucketing

When values are continuous (durations, sizes, counts), convert them to categorical buckets.

Duration Bucketing

Instead of storing exact `duration` values as a dimension, classify into performance tiers:

Duration Range	Bucket Label
< 100ms	`fast`
100ms - 500ms	`normal`
500ms - 2s	`slow`
> 2s	`very_slow`

Size Bucketing

For response sizes or log message lengths:

Size Range	Bucket Label
< 1 KB	`small`
1 KB - 100 KB	`medium`
100 KB - 1 MB	`large`
> 1 MB	`very_large`

7. Reduction Strategy 4: Hashing for Privacy

When you need to track unique entities (users, sessions) without storing identifiable information, hash the value:

Original Field	Hashed Field	Use Case
`user.email`	`hashSha256(user.email)`	Count unique users without storing the cleartext address
`client.ip`	`hashSha256(client.ip)`	Track unique clients without storing the cleartext IP

Trade-offs

- **Cardinality**: Hashing does NOT reduce cardinality — 100,000 unique emails produce 100,000 unique hashes
- **Privacy**: Hashing hides the cleartext, but it is **pseudonymization, not anonymization**. An unsalted hash of a low-entropy value can be reversed by hashing every candidate: there are only about 4.3 billion IPv4 addresses, and email addresses can be enumerated from known lists. Prefer a secret salt, or drop the field if you do not need it
- **Truncation**: Keeping only the first 8 hex characters of the hash leaves about 4.3 billion possible values, so distinct inputs start to collide at scale and `countDistinct()` undercounts — keep the full hash if exact distinct counts matter
- **When to use**: When the field is required for `countDistinct()` but must not be stored in cleartext
- **When NOT to use**: When the goal is cardinality reduction — use grouping or removal instead

8. Metric Dimension Guardrails

When configuring metric extraction in OpenPipeline (see **OPIPE-03**), apply these guardrails:

Dimension Selection Checklist

For each dimension you plan to include in an extracted metric, ask:

1. **Is this dimension needed for the metric's use case?** If the metric drives an SLO by service, you need the service field — you probably do not need `k8s.pod.name`.

2. **How many unique values does this dimension have?** Query `countDistinct()` on the field.
3. **Will this dimension grow unbounded?** Pod names, session IDs, and request IDs grow with traffic. Avoid these.
4. **Can this dimension be normalized first?** URL paths, pod names, and status codes can all be bucketed.

Cardinality Budget

Assign a cardinality budget to each extracted metric. These targets are community practice, not product limits — tune them to your tenant:

Metric Type	Target Cardinality	Max Dimensions
SLO metric (request count, error rate)	< 1,000	2-3 (service, route)
Operational metric (latency by service)	< 10,000	3-4 (service, route, status code)
Exploratory metric (for dashboards)	< 50,000	4-5 (include namespace, environment)

Breaking (SaaS 1.346): a per-record size limit on sorting and grouping keys

Cardinality has always been about the *number* of distinct values. SaaS 1.346 adds a constraint on their *size*. Verbatim: *"We now enforce a per-record size limit on field values used as sorting or grouping keys in DQL queries (such as `sort`, `summarize`, and `makeTimeseries` with `by:`)."* and *"Queries where sorting/grouping key values exceed the limit on one or more records will return a client-side error. Queries with appropriately-sized keys are unaffected and benefit from improved performance."*

This is a **query-time** limit, distinct from every ingest-time guardrail above, and it fails in a way worth anticipating:

- **One oversized record fails the whole query.** The limit is per record, so a single outlier — a full stack trace, a serialized payload, a giant URL — is enough to error a query that ran yesterday. The data is not rejected; the *query* is.

- **It is a client-side error, not a silent truncation.** That is the good outcome: you get told. Contrast the cardinality traps above, which degrade quietly.
- **The fix is the normalization you should already be doing.** Grouping by an unbounded free-text field was a cardinality problem before it was a size problem — §5 (value normalization), §6 (bucketing), and §7 (hashing) all produce compact keys as a side effect. Hashing in particular converts an arbitrarily long value into a fixed-width one.
- **Group by an extracted field, not the raw one.** `summarize count(), by:{content}` is the shape at risk; `by:{status_code}` or a parsed, normalized field is not.

The upside is real — Dynatrace notes conforming queries *"benefit from improved performance"* — but the migration cost lands on exactly the ad-hoc queries and saved tiles that group by raw text. SaaS 1.346 began its **staged tenant rollout on 08/25/2026**; until it reaches your tenant such queries keep working, which makes this a good window to find them deliberately rather than discover them through a broken dashboard.

```
// Estimate cardinality for a proposed metric extraction
// Example: request count by dt.service.name x http.route x status_code
fetch spans, from:-1h
| filter span.kind == "server" and isNotNull(endpoint.name)
| summarize unique_combos = count(), by:{dt.service.name, http.route,
http.response.status_code}
| summarize total_cardinality = count()
| fieldsAdd assessment = if(total_cardinality &lt; 1000, then: "OK: Low
cardinality",
    else: if(total_cardinality &lt; 10000, then: "CAUTION: Medium
cardinality",
    else: "WARNING: High cardinality — consider reducing dimensions"))
```

Summary

In this notebook you learned:

- **Cardinality** is the product of unique values across all dimensions — it grows multiplicatively
- **High-cardinality fields** differ by scope: `trace.id` and `http.url` for spans, `content` and `k8s.pod.name` for logs
- **Detection** — Use `countDistinct()` queries to find cardinality hotspots before they cause problems

- **Four reduction strategies**: attribute removal, value normalization, grouping/bucketing, and hashing
- **Metric guardrails** — Assign cardinality budgets and limit dimensions to what each metric's use case requires

---

Next Steps

Continue to **OPIPE-05: Business & Security Event Pipelines** to configure OpenPipeline for business transaction tracking, security event processing, and compliance-driven event routing.

References

- Metric ingestion protocol (DT docs)
- OpenPipeline processing (DT docs)
- Metric limits (DT docs)
- What's new in Dynatrace SaaS 1.346 (DT docs) — the sorting/grouping key size limit quoted in §8
- Grail data model (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.